Getting Started with Claude Code

The Complete Beginner's Guide for React.js Developers

Created by Yogesh Chavan

yogeshchavan.dev

https://www.linkedin.com/in/yogesh-chavan97/

Table of Contents

Section 1: Introduction to Claude Code

1. What is Claude Code? 4
2. Why Claude Code for React Developers? 4
3. Claude Code vs Traditional AI Assistants 5
4. Prerequisites and Requirements 6
5. Pricing and Plans 6

Section 2: Installation & Setup

1. Installing Claude Code on macOS/Linux 8
2. Installing Claude Code on Windows 8
3. Alternative Installation Methods 9
4. Logging In and Authentication 9
5. Starting Your First Session 10

Section 3: Core Concepts

1. Understanding the Agentic Loop 11
2. The Context Window Explained 12
3. Permission Modes 12
4. Built-in Tools 13

Section 4: Basic Commands & Usage

1. Essential CLI Commands 14
2. Exploring Your Codebase 15
3. Making Code Changes 16
4. Working with Git 16

Section 5: CLAUDE.md & Memory System

1. What is CLAUDE.md? 17
2. Creating Effective Instructions 18
3. Auto Memory Feature 19

4. Project Rules with .claude/rules/ 20

Section 6: Common Workflows for React Developers

1. Building React Components 21
2. Debugging React Applications 22
3. Writing Tests 23
4. Refactoring Code 24

Section 7: Advanced Features

1. Plan Mode for Complex Tasks 25
2. Using Subagents 26
3. MCP (Model Context Protocol) 27
4. Hooks and Skills 28

Section 8: Best Practices & Tips

1. Effective Prompting Techniques 29
2. Managing Context Efficiently 29
3. Common Mistakes to Avoid 30
4. Next Steps 30

Section 9: VS Code Integration

1. Installing the VS Code Extension 32
2. Opening Claude Code in VS Code 32
3. Key VS Code Features 33
4. Working with Selections 33
5. Keyboard Shortcuts 34

Section 10: Building a Complete React App

1. Project Setup 35
2. Creating the Data Model 35
3. Building Components 36
4. Custom Hooks 37
5. Testing and Debugging 38

Section 11: CI/CD Integration

1. GitHub Actions Integration 39
2. Automated PR Creation 39
3. GitLab CI/CD Integration 40
4. Scheduled Tasks 40

Section 12: Working with APIs and Backend

1. API Integration Patterns 41
2. Custom Hooks for Data Fetching 41
3. React Query Integration 42
4. Authentication Flows 43

Section 13: State Management

1. React Context Setup 44
2. Zustand Store 44
3. Redux Toolkit Setup 45
4. State Migration 45

Section 14: Performance Optimization

1. Identifying Performance Issues 46
2. Memoization Strategies 46
3. Code Splitting 47
4. Virtual Lists and Image Optimization 47

Section 15: Troubleshooting Common Issues

1. Installation Issues 48
2. Authentication Problems 48
3. Context Window Issues 49
4. Getting Help 49

Section 16: Quick Reference & Cheat Sheet

1. Essential Commands 50
2. Keyboard Shortcuts 50
3. File Locations 51
4. Common Prompts for React 51

1. Introduction to Claude Code

What is Claude Code?

Claude Code is an agentic coding tool developed by Anthropic that lives in your terminal, understands your codebase, and helps you code faster. Unlike traditional AI assistants that simply answer questions and wait for your next prompt, Claude Code can autonomously read your files, run commands, make changes, and work through complex problems while you watch, redirect, or step away entirely.

Think of Claude Code as having a highly capable junior developer working alongside you. You describe what you want in plain language, and Claude Code figures out how to build it. It explores your codebase, plans the approach, implements the solution, and verifies it works — all through natural language commands.

Claude Code is available across multiple platforms including Terminal CLI, VS Code extension, JetBrains IDEs, Desktop app, Web browser, and even Slack integration.

Why Claude Code for React Developers?

For React.js developers, Claude Code offers tremendous value by automating routine tasks and accelerating development workflows:

- **Component Generation** — Describe a component in plain language and Claude builds it with proper TypeScript types, hooks, and styling
- **Debugging** — Paste an error message and Claude traces it through your codebase, identifies the root cause, and implements a fix
- **Testing** — Claude can write comprehensive unit tests, integration tests, and fix failing tests automatically
- **Refactoring** — Modernize legacy code, convert class components to hooks, or restructure your project architecture
- **Git Operations** — Claude handles commits, branches, PRs, and merge conflicts through natural language
- **Documentation** — Generate JSDoc comments, README files, and API documentation from your code

Claude Code vs Traditional AI Assistants

Understanding how Claude Code differs from traditional AI coding assistants helps you leverage its full potential:

Feature	Traditional AI Assistants	Claude Code
Interaction Model	Question-answer format	Agentic, autonomous execution
Code Access	Copy-paste snippets	Reads entire codebase directly
File Modification	Suggests changes only	Edits files with permission
Command Execution	Not supported	Runs bash, npm, git commands
Context	Single conversation	Persistent memory across sessions
Verification	Manual by user	Auto-runs tests, lints code

Prerequisites and Requirements

Before installing Claude Code, ensure you have the following:

- **Node.js 18+** — Required for the CLI. Check with: `node --version`
- **Git** — For version control integration. Windows users need Git for Windows
- **Terminal/Command Prompt** — Any modern terminal emulator works
- **A Code Project** — A React project or any codebase to work with
- **Claude Account** — Pro, Max, Teams, Enterprise subscription, or Console account

> **Tip:** If you're new to the terminal, Claude Code documentation includes a helpful terminal guide. Don't worry — the commands are straightforward, and Claude itself can help you learn!

Pricing and Plans

Claude Code is included with Claude subscriptions. Here are your options:

- **Claude Pro ($20/month)** — Perfect for short coding sprints in small codebases with access to both Sonnet and Opus models

- **Claude Max ($100/month)** — Great value for everyday use in larger codebases with more generous usage limits
- **Claude Max ($200/month)** — Power user tier with the most access to Claude models
- **API Pay-as-you-go** — Standard Claude API pricing with no per-seat fees, ideal for teams

2. Installation & Setup

Installing Claude Code on macOS/Linux

The recommended installation method uses the native installer, which automatically keeps Claude Code updated in the background. Open your terminal and run:

```
# Native installation (recommended)
curl -fsSL https://claude.ai/install.sh | bash
```

After installation completes, you can start Claude Code immediately by running:

```
# Navigate to your project
cd your-react-project

# Start Claude Code
claude
```

Installing Claude Code on Windows

Windows users have two options depending on their terminal. First, ensure you have Git for Windows installed (download from git-scm.com if needed).

For PowerShell:

```
# Run in PowerShell (look for PS C:\ prompt)
irm https://claude.ai/install.ps1 | iex
```

For Command Prompt (CMD):

```
# Run in CMD (look for C:\ prompt without PS)
curl -fsSL https://claude.ai/install.cmd -o install.cmd
install.cmd && del install.cmd
```

> **Tip:** If you see 'The token '&&' is not a valid statement separator', you're in PowerShell, not CMD. Your prompt shows 'PS C:\' when in PowerShell. Use the PowerShell command instead.

Alternative Installation Methods

If you prefer package managers, Claude Code is also available through Homebrew and WinGet. Note that these methods do not auto-update.

Using Homebrew (macOS):

```
# Install via Homebrew
brew install --cask claude-code

# Update manually when needed
brew upgrade claude-code
```

Using WinGet (Windows):

```
# Install via WinGet
winget install Anthropic.ClaudeCode

# Update manually when needed
winget upgrade Anthropic.ClaudeCode
```

Logging In and Authentication

When you first run `claude`, you'll be prompted to log in. Claude Code supports multiple authentication methods:

- **Claude Pro/Max/Teams/Enterprise** — Recommended for most users, provides seamless access
- **Claude Console (API)** — Uses pre-paid credits, auto-creates a 'Claude Code' workspace for cost tracking
- **Third-party Providers** — Amazon Bedrock, Google Vertex AI, or Microsoft Foundry for enterprise

```
# Start Claude Code (prompts for login on first use)
claude

# Or explicitly log in
/login
```

Your credentials are stored securely on your system, so you won't need to log in again. To switch accounts later, use the `/login` command.

Starting Your First Session

Navigate to any React project directory and start Claude Code:

```
cd /path/to/your/react-project
claude
```

You'll see the Claude Code welcome screen with your session information, recent conversations, and latest updates. Try these first commands to explore:

```
# Ask about your codebase
what does this project do?

# Get help
/help

# See available commands
?
```

3. Core Concepts

Understanding the Agentic Loop

Claude Code operates through an 'agentic loop' — a cycle where Claude receives your request, decides what actions to take, executes them, observes results, and continues until the task is complete or needs your input. This is fundamentally different from simple question-answer AI.

Here's how the agentic loop works:

- **1. Receive Request** — You describe what you want in natural language
- **2. Analyze & Plan** — Claude examines your codebase and determines what needs to be done
- **3. Execute Action** — Claude uses tools (read files, run commands, edit code)
- **4. Observe Results** — Claude sees the output and evaluates success
- **5. Iterate or Complete** — Claude continues fixing issues or reports completion

This loop continues autonomously until Claude completes the task, encounters an error, or needs permission for a potentially risky action. You can interrupt at any time with **Esc**.

The Context Window Explained

The context window is Claude's working memory — it holds everything Claude knows about the current conversation including your messages, files read, command outputs, and Claude's responses. Claude's context window is approximately 200,000 tokens.

Understanding context is crucial because:

- Performance can degrade as context fills up — Claude may forget earlier instructions
- Each file read and command output consumes tokens
- Use `/clear` between unrelated tasks to reset context
- Use `/compact` to summarize and free space during long sessions

> **Tip:** Context management is the single most important skill for effective Claude Code usage. Clear your context between unrelated tasks, and be specific to avoid Claude reading unnecessary files.

Permission Modes

Claude Code has three permission modes that control how much autonomy Claude has:

- **Normal Mode** — Default mode. Claude asks permission for file writes, bash commands, and other potentially risky actions. Safe but requires more interaction.
- **Auto Mode** — A classifier reviews commands and blocks risky ones, while allowing routine operations to proceed. Use `Shift+Tab` to toggle.
- **Plan Mode** — Claude only reads and analyzes code, never modifies anything. Perfect for exploration and planning complex changes safely.

Built-in Tools

Claude Code comes with powerful built-in tools that it uses automatically:

Tool	Description	Example Use
Read	Read file contents	Analyzing your components
Edit	Modify files with permission	Updating your code
Write	Create new files	Generating new components
Bash	Execute shell commands	Running npm, git, tests
Grep	Search file contents	Finding code patterns
Glob	Find files by pattern	Locating all .tsx files
Web	Search and fetch from web	Looking up documentation

You don't need to explicitly invoke these tools — Claude uses them automatically based on what your request requires. For example, asking 'add a button to the Header component' triggers Claude to read the Header file, edit it, and potentially run your linter.

4. Basic Commands & Usage

Essential CLI Commands

These are the most important commands for daily use with Claude Code:

Command	Description	Example
claude	Start interactive mode	claude
claude "task"	Run a one-time task	claude "fix the build"
claude -p "query"	Run query, then exit	claude -p "explain this"
claude -c	Continue last conversation	claude -c
claude -r	Resume a previous session	claude -r
claude commit	Create a Git commit	claude commit
/clear	Clear conversation history	/clear
/help	Show available commands	/help
/compact	Compress context	/compact
Esc	Stop current action	Press Esc key

Interactive vs Non-Interactive Mode:

```
# Interactive mode - start a conversation
claude

# Non-interactive mode - single task
claude -p "list all React components in src/"

# Continue most recent conversation
claude --continue

# Resume specific session by name
claude --resume auth-refactor
```

Exploring Your Codebase

When you join a new project or need to understand unfamiliar code, ask Claude directly:

```
# Get a high-level overview
what does this project do?

# Understand the architecture
explain the main architecture patterns used here

# Find specific functionality
where is the authentication handled?

# Trace execution flow
trace the user login process from frontend to database

# Ask about specific files
explain what @src/components/Header.tsx does
```

Tip: Use @ to reference specific files directly. For example, '@src/App.tsx' includes the file content immediately without Claude needing to search for it, saving context and time.

Making Code Changes

Claude Code can modify your files directly. Here's how to make changes effectively:

```
# Simple addition
add a loading spinner to the UserList component

# Specific modification
add input validation to the email field in the signup form

# Bug fixes
fix the bug where clicking submit twice sends duplicate requests

# Refactoring
convert the UserCard class component to a functional component with hooks
```

When Claude proposes changes, it shows you a diff of what will be modified. You can:

- **Accept** — Allow the change and continue
- **Reject** — Decline and provide feedback
- **Accept All** — Enable auto-accept for the session

- **Esc** — Stop and reconsider the approach

Working with Git

Claude Code integrates seamlessly with Git for version control operations:

```
# Check status
what files have I changed?

# Create commits
commit my changes with a descriptive message

# Branch operations
create a new branch called feature/user-auth

# View history
show me the last 5 commits

# Handle conflicts
help me resolve the merge conflicts

# Create PRs
create a pull request for my changes
```

> **Tip:** Claude Code works best when you have the 'gh' CLI installed for GitHub operations. It enables direct PR creation, issue management, and more without rate limits.

5. CLAUDE.md & Memory System

What is CLAUDE.md?

CLAUDE.md is a special markdown file that Claude reads at the start of every session. It contains persistent instructions, project conventions, and context that Claude should always know about. Think of it as onboarding documentation specifically for Claude.

You can place CLAUDE.md files in several locations:

- **Project root (./CLAUDE.md)** — Shared with team via Git, contains project-specific instructions

- **Home folder (~/.claude/CLAUDE.md)** — Personal preferences that apply to all your projects
- **Child directories** — Loaded when Claude works with files in those directories

Generate a starter CLAUDE.md automatically:

```
# Generate CLAUDE.md based on your project
/init
```

The `/init` command analyzes your codebase and creates a file with build commands, test instructions, and conventions it discovers.

Creating Effective Instructions

A well-crafted CLAUDE.md significantly improves Claude's effectiveness. Here's an example for a React project:

```
# Project: React Dashboard

## Commands
- npm run dev: Start development server
- npm run test: Run Jest tests
- npm run lint: Run ESLint
- npm run build: Production build

## Code Style
- Use functional components with hooks, never class components
- Use TypeScript strict mode
- Prefer named exports over default exports
- Use 2-space indentation

## Architecture
- Components in src/components/
- Hooks in src/hooks/
- API calls in src/services/
- State management with Zustand in src/stores/

## Testing
- Write tests for all new components
- Use React Testing Library, not Enzyme
- Run tests before committing
```

> **Tip:** Keep CLAUDE.md concise — aim for under 200 lines. Long files consume context and reduce instruction adherence. Only include information Claude can't infer from your code.

Auto Memory Feature

Auto Memory lets Claude accumulate knowledge across sessions without you writing anything. As Claude works, it saves notes for itself: build commands, debugging insights, architecture notes, and workflow habits.

Auto Memory is enabled by default and stores files in `~/.claude/projects/<project>/memory/`.

```
# View and manage memory
/memory

# Toggle auto memory on/off
/memory  # Then use the toggle option

# Tell Claude to remember something
remember that the API tests require a local Redis instance
```

The first 200 lines of MEMORY.md are loaded at the start of every conversation. Claude keeps it concise by moving detailed notes into separate topic files.

Project Rules with .claude/rules/

For larger projects, organize instructions into topic-specific files in `.claude/rules/`. These rules can even be scoped to specific file types: